

Mechatronic Systems: Bare-Metal Architecture & Haptic Interface Design

Target Platform: Renesas RA4M1 (ARM Cortex-M4, 48 MHz) — Arduino Uno R4 Minima

Application Focus: High-Performance 1-DOF Haptic Virtual Wall Rendering

Topic Overview

This set of advanced study notes covers the synthesis of bare-metal embedded programming and mechatronic control loop design. High-performance haptic rendering requires deterministic execution, minimal control loop latency, and zero timing jitter. By bypassing abstract software layers (such as the standard Arduino core) and directly manipulating microcontroller registers, we maximize execution speed and unlock precise hardware features. These notes detail how to configure digital I/O, control advanced hardware timers, safeguard shared variables, and filter encoder data to render a stable virtual environment.

Core Concepts

- **Memory-Mapped I/O:** The paradigm where hardware peripherals share the same address space as regular system memory.
- **Direct Register Manipulation:** Modifying microcontroller configuration registers at the bit level to achieve single-cycle execution speeds.
- **Hardware Timer PWM Generation:** Utilizing specialized clock counters to generate high-frequency control signals independent of main CPU code execution.
- **Haptic Rendering Loop:** A high-speed, discrete-time control cycle (1 kHz) that tracks position, estimates velocity, filters noise, and commands motor forces to simulate physical constraints.
- **Passivity & Stability:** Graphical/haptic control rules ensuring that a digital system does not generate artificial energy, which causes dangerous oscillations.

Detailed Notes

Section 1: Memory-Mapped I/O & Bare-Metal Foundations

Traditional programming abstractions like `digitalWrite()` prioritize cross-platform compatibility over speed. They run background lookups, pin-mapping validations, and error checks, adding latency and execution variance (jitter).

The ARM Cortex-M4 processor inside the Renesas RA4M1 utilizes Memory-Mapped I/O. The processor accesses a unified 4 GB memory address space (2^{32} addresses). In this architecture, specific memory addresses are directly hardwired to physical silicon peripheral sub-systems (such as timers, ADCs, and GPIO ports).

- A **Register** is a dedicated 32-bit slot located at one of these peripheral addresses.

- Writing a 1 or 0 to a specific bit flips a physical hardware switch (transistor) on the silicon die, instantly changing an electrical state.
- In C/C++, these hardware addresses are represented as structured pointers (e.g., R_PORT1 -> PDR), allowing direct interactions with the chip's internal layout.

Section 2: The Bitmasking Toolkit

Registers are 32 bits wide, and a single I/O register controls up to 32 independent physical pins. To modify a single pin's state without corrupting or modifying adjacent pins, bitmasking via bitwise operators is required.

Core Bitwise Operations

- **Left Shift (<<):** Positions a bit mask at the target pin location. ($1 \ll 11$) shifts a binary 1 eleven places to the left, creating a bit pattern aligned perfectly with Pin 11.
- **Bitwise OR (|):** Forces a specific bit to become 1 without changing other values. Used to turn pins HIGH or configure them as outputs.
- **Bitwise AND (&):** Isolates specific bits to safely inspect their current state while ignoring background register data.
- **Bitwise NOT (~):** Inverts all bits in a sequence (1 becomes 0, 0 becomes 1). When combined with an AND assignment ($\&= \sim$), it clears targeted bits to 0 without altering adjacent data.

```
Old Register State: [Bit 31] ... [Pin 3] [Pin 2] [Pin 1] [Pin 0]
Inverted Mask (~):  1    1 ...    1    0    0    1    1
-----
New Register State: [Saved]... [Saved]  0    0  [Saved] (Pins 1 and 2 are cleared)
```

Section 3: RA4M1 Digital I/O Configuration & Board Mapping

The physical pins routed on the Arduino Uno R4 Minima board do not correlate directly with the native internal port and pin designations of the RA4M1 silicon. Direct port manipulation requires mapping individual board pins back to their native silicon locations.

Arduino Board Pin	RA4M1 Native Pin	Project Function	Register Block
Digital Pin 2	P301 (Port 3, Pin 1)	Encoder Channel A (CLK)	R_PORT3
Digital Pin 3	P302 (Port 3, Pin 2)	Encoder Channel B (DT)	R_PORT3
Digital Pin 4	P111 (Port 1, Pin 11)	Motor Driver IN1 (Direction)	R_PORT1
Digital Pin 5	P102 (Port 1, Pin 2)	Motor Driver ENA (PWM Speed)	R_PORT1
Digital Pin 7	P103 (Port 1, Pin 3)	Motor Driver IN2 (Direction)	R_PORT1

The Primary Hardware I/O Registers

- **PDR (Port Direction Register):** Dictates electrical direction. Setting a bit to 1 creates an Output (power source/sink); setting it to 0 creates an Input (high-impedance sensing).
- **PODR (Port Output Data Register):** Manages the output state of pins configured in the PDR. Writing a 1 puts 3.3V on the pin; writing a 0 ties it to ground.
- **PIDR (Port Input Data Register):** A read-only register that takes a snapshot of the live logic voltages present on the physical input pins.

Section 4: Hardware Timers & Proportional PWM Generation

To deliver smooth force commands without stalling the main processor, the system programs General Purpose Timer 2 (GPT2) to manage hardware-driven Pulse Width Modulation (PWM). The RA4M1 chip features an internal master clock frequency of 48 MHz.

Critical Timer Registers

- **GTPR (General Purpose Timer Period Register):** Sets the maximum value the timer's internal stopwatch counts up to before rolling over to zero. This ceiling directly establishes the final PWM carrier frequency.
- **GTCNT (General Purpose Timer Counter Register):** The active stopwatch register, ticking upward from 0 to the GTPR value at a rate of 48 million steps per second.
- **GTCCR (General Purpose Timer Compare Capture Register):** Sets the electrical tripwire (Duty Cycle). The PWM pin turns ON when GTCNT resets to 0, and switches OFF the exact instant GTCNT matches the value stored in GTCCR.

Adaptive Scaling Formula

To sweep through varying PWM frequencies without distorting the mechatronic force commands, the code converts a standard 8-bit input variable (`pwmValue` from 0–255) into an adaptive hardware compare limit via the following scaling calculation:

$$GTPR = 48,000,000 / \text{Desired Frequency (Hz)}$$

$$\text{compareValue} = \text{pwmValue} \times (GTPR / 255.0)$$

Section 5: Interrupt Architecture & Data Atomicity

Encoder positioning signals trigger External Hardware Interrupts, pausing the main execution loop instantly to update the `encoderPos` tracking variable. Because the RA4M1 is a 32-bit architecture, reading or writing a large variable requires a specific instruction cycle. If an interrupt executes precisely while the main control loop is reading `encoderPos`, the data can split across cycles, corrupting the position reading (Data Race Condition).

Best Practice: Atomic Guards

To protect shared variables, the control loop must enforce Atomicity by temporarily pausing the interrupt engine during critical read operations:

```
__disable_irq();           // Globally pause all interrupts
currentRawPos = encoderPos; // Copy the protected variable safely
__enable_irq();            // Re-enable interrupts immediately
```

Section 6: Haptic Control Loop Stability & The L298N Thermal Limit

A haptic virtual wall acts as a discrete-time spring-damper system. When the user pushes the master linkage past a designated virtual threshold (x_{wall}), the system calculates an opposing force vector:

$$F_{wall} = -(K_{stiffness} \times \Delta x + B_{damping} \times v)$$

Z-Width and Sampling Realities

A device's Z-Width defines the range of virtual impedances (stiffness and damping combinations) it can stably simulate without breaking out into uncontrolled oscillations. High control-loop latency or low PWM frequencies reduce this stable operating zone, making virtual boundaries feel soft, mushy, or violently erratic.

Hardware Limitation: The L298N Power Stage

While high PWM frequencies (above 20 kHz) make motor operation completely silent to human hearing, the L298N H-Bridge Driver relies on older Bipolar Junction Transistor (BJT) technology. BJTs switch states relatively slowly. Driving an L298N at ultra-high frequencies forces its internal transistors to spend too much time transitioning through their linear states, generating significant waste heat. For this driver, the optimal performance sweet spot sits between 8 kHz and 16 kHz, balancing signal smoothness against component thermal limits.

Section 7: Signal Processing & Velocity Estimation

Calculating velocity by finding the raw derivative of digital encoder counts introduces significant quantization noise. This noise manifests as sharp force spikes that ruin the haptic experience. To counteract this, the system processes raw velocity commands through a digital 2nd-Order Butterworth Infinite Impulse Response (IIR) Filter:

$$y[n] = b_0x[n] + b_1x[n-1] + b_2x[n-2] - a_1y[n-1] - a_2y[n-2]$$

This recursive digital filter structure suppresses high-frequency encoder noise, smoothing the calculated velocity profile while maintaining low phase lag for the active control loop.

Definitions

- **Bitmasking:** A software technique that uses bitwise logical operators (AND, OR, NOT, XOR) to manipulate individual bits within a wider binary byte or word.
- **Jitter:** Spatial or temporal variance in the execution interval of a periodic control loop, disrupting deterministic calculations.

- **Duty Cycle:** The ratio of time an electrical signal remains active (HIGH) versus its total operational period, expressed as a percentage.
- **Z-Width:** The dynamic envelope of physical stiffness and damping characteristics a haptic peripheral can stably render without introducing instability.
- **Passivity:** A safety design condition where a system cannot output more total kinetic energy than an operator puts into it, ensuring absolute stability.
- **H-Bridge:** A four-switch switching network configuration that enables DC voltages to be applied across a load or motor winding in either direction.
- **Atomicity:** An operation sequence that executes completely without interruption or division, preventing data corruption across concurrent threads or interrupts.

Important Formulas / Rules

1. Hardware Timer Period Counting

$$GTPR\ Value = System\ Peripheral\ Clock\ Speed\ (Hz) / Desired\ PWM\ Carrier\ Frequency\ (Hz)$$

2. Adaptive Hardware Duty Cycle Assignment

$$GTCCR\ Register\ Value = Target\ PWM\ Input\ (0\ to\ 255) \times (Current\ GTPR\ Value / 255.0)$$

3. General Virtual Wall Force Law

$$F_{wall} = -(K_{stiffness} \cdot (x_{current} - x_{wall}) + B_{damping} \cdot v_{filtered}), \text{ if } x_{current} > x_{wall}$$

$$F_{wall} = 0, \text{ if } x_{current} \leq x_{wall}$$

4. Passivity Unidirectional Constraint

$$\text{If } F_{wall} > 0 \rightarrow F_{wall} = 0 \quad (\text{Ensures the wall only pushes back, never pulls you in})$$

Examples

Complete Parameter Scaling Walkthrough

System Constants: System Clock = 48 MHz, Selected Tuning Frequency = 16 kHz, Commanded Input Force Variable (pwmValue) = 51.

Step 1: Calculate the Timer Counting Ceiling

GTTPR Register = $48,000,000 / 16,000 = 3000$ clock counts

Step 2: Compute the Hardware Tripwire Compare Register

GTCCR Register = $51 \times (3000 / 255.0) = 51 \times 11.764 = 600$ counts

Step 3: Verify the Resulting Duty Cycle Percentage

Duty Cycle = $(GTCCR / GTTPR) \times 100\% = (600 / 3000) \times 100\% = 20\%$

Direct Mapping Verification: $(51 / 255) \times 100\% = 20\%$

H-Bridge Truth Table (L298N Core)

The following table outlines how the direction control lines manipulate the state of the H-bridge drivers:

IN1 Register Bit (P111)	IN2 Register Bit (P103)	Resulting Motor Shaft State
0	0	Coast Mode (High-Impedance Open Floating Circuit)
1	0	Forward Operation (Current moves Left-to-Right)
0	1	Reverse Operation (Current moves Right-to-Left)
1	1	Brake Mode (Terminals shorted together to absorb Back-EMF)

Common Mistakes

× Conflating Board Pin Layouts with Silicon Registers

The Error: Assuming that writing $(1 \ll 4)$ to R_PORT1 will modify physical Arduino Pin 4.

The Correction: Arduino Pin 4 routes directly to Bit 11 on Port 1 (P111). Always cross-reference the official board schematics before writing bare-metal code.

× The Dual-High H-Bridge Destruction Myth

The Error: Fearing that setting both directional input control registers (IN1 and IN2) to a high logic level (1) will cause a short-circuit on the driver board.

The Correction: Setting both inputs HIGH activates the H-bridge's high-side electronic switches simultaneously. This brings both terminals of the motor to the exact same voltage level, initiating Brake Mode. The motor safely locks up to resist movement; no short circuit occurs.

× Omission of Atomic Guards on Shared Variables

The Error: Reading multi-byte encoder counts inside the primary control loop without disabling hardware interrupts.

The Correction: An asynchronous interrupt can trigger mid-read, scrambling the upper and lower bytes of data and introducing huge, sudden position errors. Always encase global interrupt variables inside an atomic block using global interrupt flag controls (`__disable_irq()`).

Interview / Viva Questions

- **Q: Why does direct register access improve virtual wall stiffness compared to standard Arduino library calls?**

A: Standard Arduino calls introduce significant overhead via safety lookups and run-time pin translation tables. This delay increases control-loop latency and introduces timing variation (jitter). Jitter breaks the deterministic execution required for discrete-time control loops, causing the system to add artificial energy. Direct register writes execute in single clock cycles, eliminating jitter and maximizing the stable stiffness limit ($K_{\text{stiffness}}$) of the virtual wall.

- **Q: Why shouldn't you run an older L298N driver block at a standard ultrasonic frequency like 24 kHz?**

A: The L298N is a Bipolar Junction Transistor (BJT) design, which switches states significantly slower than modern MOSFETs. At high frequencies like 24 kHz, the transitions between the fully ON and fully OFF states occupy a major percentage of the total operating period. Because the transistors spend so much time in this high-resistance linear region, they convert large amounts of power into waste heat, leading to rapid thermal breakdown.

- **Q: What is the mechanical role of the digital Butterworth filter in this system architecture?**

A: Optical encoder units track positional transitions down to discrete counts. Differentiating this step-wise positional data to compute velocity introduces severe high-frequency step noise. If left unfiltered, this noise passes directly into the motor commands, manifesting as violent acoustic buzzing and physical vibrations. A 2nd-order Butterworth filter attenuates this high-frequency noise while minimizing phase lag, keeping the control loop stable and responsive.

Quick Revision Sheet (1-Page Summary)

BARE-METAL MECHATRONICS REVISION CHEAT SHEET

[MEMORY MAP PERIPHERALS]

- * Every hardware feature is a memory address block.
- * R_PORTX -> Handles GPIO Pins.
- * R_GPTX -> Handles Advanced Counter/Timer Control.

[BIT MANIPULATION TOOLKIT]

- * Mask Set (HIGH / Output): $REG \mid= (1 \ll PIN_BIT)$
- * Mask Clear (LOW / Input): $REG \&= \sim(1 \ll PIN_BIT)$
- * Pin Isolation Check: $if (INPUT_REG \& (1 \ll PIN_BIT))$

[RA4M1 REGISTERS (ARDUINO UNO R4 MINIMA)]

- * PDR -> Direction Selection Register (1 = Output, 0 = Input)
- * PODR -> Digital Output Drive Register (1 = 3.3V, 0 = 0V)
- * PIDR -> Input Voltage Capture Register (Reads live pin states)

[HARDWARE TIMER CONFIGURATION (GPT2)]

- * System Master Clock Speed: 48 MHz
- * GTPR -> Stores the counting ceiling to define the PWM Carrier Frequency.
- * GTCCR -> Stores the tripwire value to define the Duty Cycle (Motor Torque).
- * Variable Tuning Rule:
 $Ceiling\ Counts = 48,000,000 / Target_Frequency_Hz$
 $Compare\ Register\ Value = Duty_Cycle_Integer * (Ceiling\ Counts / 255.0)$

[DATA ATOMICITY RULE]

- * Never read an interrupt-updated variable inside your main control loop without enclosing it in a protective interrupt lock block:
--> `__disable_irq(); [Read/Write Operation] __enable_irq();`

[HAPTIC WALL CONTROL LOGIC]

- * Spring-Damper Feedback Law: $Force = -(K * Penetration_Depth + B * Velocity)$
- * Passivity Check: Force command values must be clipped to zero if the sign inverts, preventing energy injection.
- * Ideal L298N Driver Operating Window: 8 kHz to 16 kHz to prevent overheating.

Final Takeaways

1. **Registers Drive the Hardware:** Direct register manipulation bypasses slow software abstraction layers, turning multi-cycle lookups into rapid, single-cycle physical operations.
2. **Frequency Controls Quality:** High-frequency PWM smooths out current ripple and eliminates motor noise, but must be balanced against the switching speeds and thermal limits of your driver electronics (8–16 kHz for the L298N).
3. **Protect Shared Variables:** In asynchronous systems, atomic guards are essential. Unprotected multithreaded or interrupt-driven variables will eventually suffer from data race conditions, leading to unexpected system glitches.

4. **Stability Demands Determinism:** High-fidelity mechatronic rendering relies on constant execution intervals. Minimizing timing jitter is the single most effective way to optimize control loop performance and build highly realistic virtual environments.